Sniplink — URL Shortener (Full-Stack Knowledge Base)

1. Project Overview

Sniplink is a full-stack URL shortener that lets users create short, memorable links, track click analytics, set expiry dates, generate QR codes, and use custom aliases. It has a simple username/password auth system, Redis caching for fast redirects, and MongoDB for persistent storage.

Brand Name: Sniplink
Project Name in Code: `vite_react_shadcn_ts`
Repo Root: `/Users/aryamanagarwal/Desktop/vs_code_Aryaman/Projects/url`

---

2. Tech Stack

Frontend

Technology	Purpose
React 18	UI library
Vite 5	Build tool & dev server
TypeScript	Type safety
Tailwind CSS 3	Utility-first styling
shadcn/ui	Component library (Radix-based)
React Router v6	Client-side routing
TanStack React Query	Server state management
Recharts	Charts (available but not used yet)
Axios	HTTP client
react-hook-form + zod	Form validation (available)
lucide-react	Icons
qrcode.react	QR code generation
sonner + shadcn toast	Toast notifications
clsx + tailwind-merge	Classname utilities
date-fns	Date manipulation (available)

Backend

Technology	Purpose
Flask 3.1	Python web framework
PyMongo 4.16	MongoDB driver

Redis 7.4	Caching & rate limiting
itsdangerous	Auth token signing
Werkzeug	Password hashing (generate_password_hash, check_password_hash)
flask-cors	CORS support
python-dotenv	Environment variable loading
Gunicorn 25	WSGI server (production)

Testing

Technology	Purpose
Vitest	Unit/integration tests
jsdom	DOM environment for tests
@testing-library/react	Component testing (available)
Playwright	E2E tests

Infrastructure

Service	Purpose
Render	Flask backend hosting
Vercel	Frontend static hosting
MongoDB Atlas	Database
Redis (Render or self-hosted)	Cache

---

3. System Design & Architecture

```
┌─────────────┐     ┌──────────────┐     ┌─────────┐
│  Browser /  │ ──▶ │  Vite Dev    │ ──▶ │  Flask  │
│  Vercel     │     │  Server      │     │  API    │
│  (React SPA)│     │  (:8080)     │     │  (:5000)│
└─────────────┘     └──────────────┘     └────┬────┘
                                              │
                                   ┌──────────┴──────────┐
                                   │                     │
                              ┌────▼─────┐         ┌─────▼─────┐
                              │ MongoDB  │         │   Redis   │
                              │ (Atlas)  │         │  (Cache + │
                              │          │         │  Rate Lim)│
                              └──────────┘         └───────────┘
```

Request Flow:

1. User opens React SPA → served from Vite dev server (port 8080) or Vercel
2. API calls (`/api/*`) are proxied to Flask in dev via `vite.config.ts` proxy
3. Flask authenticates (Bearer JWT-like tokens via itsdangerous), rate-limits (Redis), and queries MongoDB
4. Redirects (`GET /<short_code>`) check Redis cache first, fall back to MongoDB, cache on miss
5. Built frontend (`dist/`) can be served directly by Flask for single-service deployment

---

4. Backend Deep Dive

4.1 File: `backend/app.py` (422 lines)

Auth System

- **Token:** URLSafeTimedSerializer (itsdangerous) — not standard JWT, but a signed timed token
- **Secret:** From `AUTH_SECRET` env var, falls back to `SECRET_KEY` , then `"dev-secret"`
- **Expiry:** 7 days default (`AUTH_TOKEN_MAX_AGE_SECONDS = 604800`)
- **Storage:** User documents in MongoDB `users` collection with `_id = username` , `password_hash` with Werkzeug

Endpoints

Method	Route	Auth	Description
POST	`/api/register`	No	Create account, returns token
POST	`/api/login`	No	Login, returns token
POST	`/api/shorten`	Yes (Bearer)	Create short URL
GET	`/api/urls`	Yes (Bearer)	List user's URLs (paginated)
GET	`/api/stats/<code>`	No	Get click stats for a short code
GET	`/<short_code>`	No	Redirect to original URL
GET	`/`, `/login`, `/register`	No	Serve frontend or API info
GET	`/assets/<path>`	No	Serve built frontend assets

Rate Limiting (Redis)

- Key: `rate_limit:{user_ip}`
- Window: 60 seconds
- Limit: 5 requests per window
- Implemented only on `/api/shorten`

Caching (Redis)

- On redirect, check Redis cache first
- Cache key: `short_code` → original URL
- TTL: 3600 seconds (1 hour)
- Clicks are incremented even on cache hit

URL Shortening Logic

1. Check for existing URL with same `(user_id, original_url, expiry)` tuple — dedup
2. If `custom_alias` provided, check uniqueness

3. Otherwise, generate random 6-char alphanumeric code (62^6 = ~56B combinations)
4. Collision detection: loop until unique
5. Compound unique index on `(user_id, original_url, expiry)` enforces DB-level dedup

Expiry Handling

- Optional ISO datetime string from frontend
- If expiry is in the past, the redirect returns HTTP 410 (Gone)
- Stored as datetime in MongoDB

CORS

- Configurable via `CORS_ORIGINS` env var (comma-separated or `*`)
- Applied only to `/api/*` routes
- Headers: `Content-Type`, `Authorization`

Graceful Degradation

- MongoDB connection failure: returns 503 on DB-dependent endpoints
- Redis failure: silently disables caching & rate limiting (try/except, sets `redis_client = None`)
- DB connection timeout: 3000ms `serverSelectionTimeoutMS`

4.2 File: `backend/requirements.txt`

```
blinker, click, dnspython, Flask==3.1.3, flask-cors==6.0.2, gunicorn==25.2.0,
itsdangerous==2.2.0, Jinja2, MarkupSafe, packaging, pymongo==4.16.0,
python-dotenv==1.2.2, redis==7.4.0, Werkzeug==3.1.7
```

4.3 File: `backend/Procfile`

```
web: gunicorn app:app
```

4.4 File: `backend/data/urls.json`

Sample document structure for export/backup:

```json
{
  "short_code": "abc123",
  "original_url": "https://google.com",
  "user_id": "user123",
  "expiry": "2026-03-30",
  "clicks": 0
}
```

---

5. Frontend Deep Dive

5.1 Routing (`src/App.tsx`)

```
/           → Index (RequireAuth wrapper — redirects to /login if no token)
/login      → Login
/register   → Register
*           → NotFound
```

5.2 Pages

`Index.tsx` — Main Dashboard

- Header with logo ("Sniplink"), ThemeToggle, Logout button
- URL shortener form (`ShortenForm`)
- Result display with QR code (`ResultCard`)
- Link analytics lookup (`StatsCard`)
- "My Links" table listing all user's URLs (fetched from `GET /api/urls`)
- Table columns: Short Code, Original URL, Clicks, Expiry
- Manual "Refresh" button (no auto-refetch)

`Login.tsx`

- Username + password form
- On success: stores token + username in localStorage, toast notification, navigate to `/`
- Redirects to `/` if already authenticated (in `useEffect`)

`Register.tsx`

- Username + password form
- On success: auto-logs in (stores token), toast, navigate to `/`
- Redirects to `/` if already authenticated

`NotFound.tsx`

- Simple 404 page with link back to `/`

5.3 Components

`ShortenForm.tsx`

- URL input (type=url, required)
- "Advanced options" collapsible: Custom alias text input, Expiry date picker (IOSDatePicker)
- Submit calls `shortenUrl()` API, passes result up via `onResult` callback
- Toast on success/error
- Clears form after successful shortening

`ResultCard.tsx`

- Displays created short URL (clickable link)
- Copy to clipboard button (with checkmark feedback, 2s timeout)
- Open in new tab button
- QR code via `qrcode.react` (SVG)
- Download QR as SVG button (serializes SVG DOM element to blob, triggers download)

`StatsCard.tsx`

- Short code input with "Lookup" button
- Calls `GET /api/stats/<code>` (public, no auth needed)
- Displays: Original URL, Click count, Expiry date

`ThemeToggle.tsx`

- Sun/Moon toggle button
- Persists to `localStorage.theme`
- Respects system preference via `prefers-color-scheme` media query on initial load
- Toggles `dark` class on `<html>`

`IOSDatePicker.tsx`

- Custom wheel-style date/time picker inspired by iOS
- Uses shadcn Drawer (Vaul) for mobile-friendly bottom sheet
- 5 columns: Month, Day, Year, Hour, Minute (5-min intervals)
- Each column is a `WheelColumn` with snap-scroll behavior
- Gradient fade edges, highlight bar, snap-to-item on scroll end (debounced 80ms)
- Confirm/Cancel buttons
- Outputs ISO datetime string (truncated to minute)

`NavLink.tsx`

- Thin wrapper around React Router's `NavLink` with `className` + `activeClassName` + `pendingClassName` props

5.4 Lib

`api.ts`

- Axios instance with `VITE_API_BASE_URL` base
- Request interceptor: auto-attaches `Bearer` token from localStorage
- Typed interfaces: `AuthRequest`, `AuthResponse`, `ShortenRequest`, `ShortenResponse`, `StatsResponse`, `UrlItem`, `UrlsListResponse`
- Functions: `shortenUrl()`, `registerUser()`, `loginUser()`, `getStats()`, `listUrls()`

`auth.ts`

- `TOKEN_KEY = "sniplink_token"`, `USERNAME_KEY = "sniplink_username"`
- `getAuthToken()` / `getAuthUsername()` — read from localStorage
- `setAuth(token, username)` — save + set dark theme on first login
- `clearAuth()` — remove from localStorage

`utils.ts`

- `cn()` — Tailwind class merge utility (clsx + twMerge)

5.5 Hooks

`use-toast.ts`

- Reducer-based toast state management (shadcn pattern)
- `TOAST_LIMIT = 1`, `TOAST_REMOVE_DELAY = 1,000,000ms`
- Exports `toast()` function and `useToast()` hook
- Listener pattern for component re-renders

`use-mobile.tsx`

- `useIsMobile()` hook based on `matchMedia("(max-width: 767px)")`

5.6 UI Components (shadcn/ui)

49 components under `src/components/ui/` including: accordion, alert-dialog, avatar, badge, button, card, chart, checkbox, command, dialog, drawer, dropdown-menu, form, input, label, popover, select, sheet, sidebar, skeleton, sonner, switch, table, tabs, toast, toggle, tooltip, etc.

5.7 Styling

- `index.css` — CSS variables (HSL) for light & dark themes
- Theme: Indigo primary (`234 89% 60%`), slate-based neutrals
- `border-radius: 0.75rem` (12px)
- Font: Inter via Google Fonts

---

6. Database Schema

MongoDB — `url_shortener` database

Collection: `urls`

```
{
  _id: ObjectId (auto),
  short_code: String (unique, indexed),
  original_url: String,
  user_id: String (references users._id),
  expiry: DateTime (optional, nullable),
  clicks: Number (default 0)
}
```

- Compound unique index: `(user_id, original_url, expiry)`
- Sorted by `clicks` descending on list

Collection: `users`

```
{
  _id: String (username),
  password_hash: String (Werkzeug hash)
}
```

- Simple key-value pattern using `_id` as username

Redis

- Cache: `{short_code} → {original_url}` (TTL: 3600s)
- Rate limit: `rate_limit:{ip} → {count}` (TTL: 60s)

---

7. Environment Variables

Variable	Default	Description
`PORT`	`5000`	Flask port
`FLASK_HOST`	`127.0.0.1`	Flask bind host
`MONGO_URI`	`mongodb://localhost:27017/`	MongoDB connection string
`MONGO_DB`	`url_shortener`	Database name
`MONGO_COLLECTION`	`urls`	Collection name
`REDIS_HOST`	`localhost`	Redis host
`REDIS_PORT`	`6379`	Redis port
`REDIS_DB`	`0`	Redis DB index
`SHORT_BASE_URL`	`request.host_url`	Public domain for short URLs

`AUTH_SECRET`	`"dev-secret"`	Token signing secret (required in prod)
`AUTH_TOKEN_MAX_AGE_SECONDS`	`604800`	Token expiry (7 days)
`CORS_ORIGINS`	`*`	Allowed CORS origins (comma-separated)
`SECRET_KEY`	—	Fallback for AUTH_SECRET
`MONGODB_URI`	—	Fallback for MONGO_URI
`VITE_API_BASE_URL`	—	Frontend: base URL for API calls
`VITE_BACKEND_URL`	—	Vite proxy target in dev
`VITE_BACKEND_PORT`	—	Alternative way to set backend port for Vite proxy

---

8. Development & Build

Scripts (`package.json`)

Script	Command
`dev`	`vite` — starts frontend on :8080
`backend:start`	`python3 app.py` — starts Flask on :5000
`backend:start:venv`	`./.venv/bin/python app.py`
`dev:full`	`npm run backend:start & npm run dev` — both servers
`build`	`vite build` — outputs to `dist/`
`build:dev`	`vite build --mode development`
`lint`	`eslint .`
`preview`	`vite preview`
`test`	`vitest run`
`test:watch`	`vitest`
`test:e2e`	Playwright E2E tests
`test:e2e:ui`	Playwright UI mode

Dev Proxy (Vite)

- `/api/*` requests proxied to Flask backend
- Target: `VITE_BACKEND_URL` → falls back to `http://127.0.0.1:{VITE_BACKEND_PORT || PORT || 5000}`

Setup

```
# Backend
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env  # edit as needed
python app.py

# Frontend
npm install
npm run dev
```

---

9. Deployment

Backend — Render

- Start command: `gunicorn app:app` (from `Procfile`)
- Set env vars: `MONGO_URI` , `REDIS_HOST` , `AUTH_SECRET` , `SHORT_BASE_URL` , `CORS_ORIGINS`

Frontend — Vercel

- Framework: Vite
- Build: `npm run build`
- Output: `dist/`
- Env var: `VITE_API_BASE_URL=https://urlshortner-1xj7.onrender.com`

Single-Service (Flask serves built frontend)

- Run `npm run build` → `dist/` is created
- Flask auto-serves `dist/index.html` at `/` and assets at `/assets/<path>`
- Useful for Render single service (no separation)

---

10. Testing

Vitest (Unit Tests)

- Config: `vitest.config.ts`
- Environment: jsdom with globals
- Setup: `src/test/setup.ts` — imports `@testing-library/jest-dom` , mocks `matchMedia`
- Test files: `src/**/*.{test,spec}.{ts,tsx}`
- Current tests: `src/test/example.test.ts` (basic placeholder)

Playwright (E2E Tests)

- Config: `tests/e2e/playwright.config.ts`
- Base URL: `http://localhost:8080` (Vite dev server)
- Fixture: `tests/e2e/playwright-fixture.ts` (empty re-export)
- 30s timeout, trace on first retry

---

11. Known Gaps & Notes

- No rate limiting on auth endpoints (only on `/api/shorten`)
- Token is not a standard JWT (itsdangerous signed dict)
- No password strength validation

- No URL validation beyond HTML5 `type=url`
- No HTTPS enforcement in code (handled by deployment)
- CORS is wide-open (`*`) by default
- "My Links" table requires manual "Refresh" click (no auto-polling or React Query cache invalidation)
- Recharts is available in dependencies but not yet used
- No CSRF protection (token-based auth mitigates this)
- clicks counter is not atomic-safe under concurrent requests in current implementation (no findOneAndUpdate)
- `base62_encode` function exists but is not used (uses random generation instead)

---

12. Interview Questions

Architecture & Design

1. **Why did you use MongoDB instead of PostgreSQL?** — For the URL shortener, MongoDB offers schema-less flexibility for documents that may have different fields (optional expiry, custom aliases), easy horizontal scaling, and simple document-based storage for key-value-like lookups. The data model is simple and doesn't require joins.

2. **How does the system handle high traffic for redirects?** — Redirects (`GET /<short_code>`) are cached in Redis with a 1-hour TTL. On cache hit, we skip DB entirely and still increment clicks via MongoDB. This reduces DB load significantly. Rate limiting (Redis, 5 req/min/IP on shorten) prevents abuse on the creation endpoint.

3. **Why use Redis for both caching and rate limiting?** — Redis is in-memory and extremely fast for both use cases. For caching, it reduces MongoDB read load. For rate limiting, Redis `INCR` + `EXPIRE` provides a simple sliding window. Using one Redis instance for both reduces infrastructure complexity.

4. **How does the deduplication of URLs work?** — Before creating a short URL, the system checks if the same `(user_id, original_url, expiry)` tuple already exists. MongoDB has a compound unique index enforcing this. If found, the existing short code is returned instead of creating a duplicate.

5. **What happens if MongoDB or Redis goes down?** — The system degrades gracefully. If MongoDB is unavailable at startup, all DB-dependent endpoints return 503. If Redis is unavailable, the system silently disables caching and rate limiting — redirects go directly to MongoDB, and rate limits are not enforced.

6. **How do you prevent short code collisions?** — Two strategies: (1) Random 6-char alphanumeric generation (62^6 = ~56 billion combinations), (2) Collision check loop that regenerates if the code already exists in MongoDB. Custom aliases are checked for uniqueness before insertion.

7. **Why itsdangerous instead of standard JWT for auth tokens?** — itsdangerous provides URLSafeTimedSerializer which is simpler for this use case — no need for public/private key pairs, no JWKS setup. It signs a dict with a timestamp and verifies expiry. For a simple username-password auth in a URL shortener, it's adequate.

8. **How would you scale this application?** — Add a CDN (Cloudflare) for edge caching of redirects, use Redis cluster for distributed caching, shard MongoDB by short_code prefix, add more Flask workers (Gunicorn), and potentially move to a distributed rate limiter.

9. **Why is the frontend built with Vite instead of Create React App?** — Vite offers faster dev startup (native ESM, no bundling), faster HMR, smaller production builds (Rollup-based), and better TypeScript support out of the box.

Backend (Flask/MongoDB/Redis)

10. **Explain the auth token verification flow.** — The `Authorization: Bearer <token>` header is extracted. The token is deserialized using `URLSafeTimedSerializer.loads()` with a max_age of 7 days. If the signature is bad or expired, it returns 401. Otherwise, the username is extracted and returned for use in the request handler.

11. **How are passwords stored?** — Passwords are hashed using Werkzeug's `generate_password_hash()` which uses PBKDF2 with a SHA-256 salt by default. Verification uses `check_password_hash()` which extracts the salt from the stored hash and re-computes.

12. **What indexes exist on the urls collection?** — A compound unique index on `(user_id, original_url, expiry)`. There is no explicit index on `short_code` or `user_id` alone (implicit due to unique constraint searches in queries).

13. **Why does the clicks counter increment even on Redis cache hits?** — Click accuracy is important for analytics. Even if the redirect is served from cache, we still update MongoDB with `$inc: {clicks: 1}` to ensure the counter reflects all visits.

14. **How does the expiry feature work?** — The frontend sends an optional ISO datetime string. The backend parses it with `datetime.fromisoformat()`. On redirect, if the expiry datetime is before `datetime.utcnow()`, it returns HTTP 410 (Gone) with a "Link expired" message.

15. **What's the difference between `/shorten` and `/api/shorten`?** — None functionally. Both routes are registered on the same handler via `@app.route()` decorator stacking. This provides backward compatibility or flexibility for different API prefix conventions.

16. **How does CORS work in this project?** — `flask-cors` is conditionally imported and applied only to `/api/*` routes. The allowed origins come from the `CORS_ORIGINS` env var, defaulting to `*`. Headers `Content-Type` and `Authorization` are explicitly allowed.

17. **Why is `load_dotenv` wrapped in try/except?** — The `python-dotenv` package is optional. If it's not installed, the app should still run using system environment variables. The try/except ensures the app doesn't crash if the package is missing.

Frontend (React/TypeScript)

18. **How does the frontend handle authentication?** — Auth token is stored in `localStorage` under `sniplink_token`. The `RequireAuth` wrapper component in `App.tsx` checks for the token on every route navigation to `/`. If absent, it redirects to `/login`. The Axios interceptor auto-attaches the token as a Bearer header on every API request.

19. **Explain the QR code generation and download flow.** — `qrcode.react` renders an SVG element. The download button serializes the SVG DOM to a string using `XMLSerializer`, creates a Blob, generates an object URL, programmatically clicks a download anchor, and revokes the URL — all in the browser, no server needed.

20. **How does the iOS-style date picker work?** — Five `WheelColumn` components (Month, Day, Year, Hour, Minute) each render a scrollable list with snap-scroll. On scroll end (debounced 80ms), the

nearest item is selected. The visible area has a highlighted bar in the center and gradient fades at the top/bottom. The picker is presented in a bottom Drawer (Vaul) for mobile-friendly interaction.

21. **Why does the "My Links" table use a manual "Refresh" button instead of auto-fetching?** — Likely a design choice to reduce API calls. However, this is a known gap — TanStack Query is available in dependencies and could be used for automatic background refetching with cache invalidation after shortening a URL.

22. **How is dark mode implemented?** — CSS variables in HSL define light and dark themes. A `dark` class on the `<html>` element toggles between them. The `ThemeToggle` component persists the preference to `localStorage.theme` and respects the system `prefers-color-scheme` on initial load.

23. **What is the purpose of the `NavLink` component?** — It wraps React Router's `NavLink` to provide a simpler API for passing `className`, `activeClassName`, and `pendingClassName` as separate props instead of using a render-prop callback pattern. It uses `forwardRef` and `cn()` utility.

24. **What would happen if two users simultaneously shortened the same URL?** — The compound unique index `(user_id, original_url, expiry)` prevents duplicate documents at the database level. If the same user attempts to shorten the same URL twice, the existing short code is returned without creating a duplicate.

General Engineering

25. **How would you add click analytics with timestamps (e.g., last 7 days of clicks)?** — Add a `clicks_log` collection with documents like `{short_code, timestamp, user_agent, referrer, ip_hash (for privacy)}`. On each redirect, insert a log entry. For stats aggregation, use MongoDB's aggregation pipeline with `$match` and `$group` by date.

26. **How would you implement a custom domain feature?** — Allow users to add verified domains in their profile. Store a CNAME record pointing the custom domain to the shortener service. On request, check the `Host` header, look up the domain-to-user mapping, and serve the appropriate short URLs. This requires a dynamic virtual hosting setup.

27. **How would you prevent malicious URL shortcode generation (e.g., phishing)?** — Add URL validation against a blocklist of known malicious domains (Google Safe Browsing API). Rate-limit creation endpoints aggressively. Require email verification for new accounts. Add abuse reporting functionality.

28. **Explain the difference between the two short code generation methods.** — `generate_short_code()` produces random alphanumeric strings (6 chars, 62^6 combinations) and checks for collisions. `base62_encode()` converts a numeric ID to a base-62 string (like YouTube), giving deterministic, shorter codes for large numbers. The random method is used; base62 is available but unused.

29. **How would you add password reset functionality?** — Add a `POST /api/forgot-password` that generates a timed token (similar to auth token) linked to the user, sends it via email (SendGrid, SES), and a `POST /api/reset-password` that verifies the token and updates the password hash. The frontend would have a ForgotPassword page collecting email.

30. **What security considerations are missing?** — Rate limiting on auth endpoints (prevent brute force), HTTPS enforcement, input sanitization beyond werkzeug, CSRF protection (partially

mitigated by token auth), password strength requirements, no refresh token rotation, no email verification, logs may expose IPs and user data.

Revisit/Review Prompts

31. **How would you refactor the backend to use async/await?** — Switch from Flask to Quart (async Flask-compatible) or FastAPI. Use `asyncpg` or `motor` (async MongoDB driver). Use `aioredis` for Redis. This would improve throughput for I/O-bound operations like DB queries and cache lookups.
32. **How does the Vite proxy work in development?** — `vite.config.ts` defines a proxy rule: any request to `/api` is forwarded to the backend target (default `http://127.0.0.1:5000`) with `changeOrigin: true`. This avoids CORS issues during development since the browser sees everything coming from the same origin.
33. **Why is there a `base62_encode` function that's never used?** — It's a planned upgrade path. Random generation works for current scale, but base62 encoding from a monotonically increasing counter would produce shorter codes, avoid collision loops, and be more scalable. The function was written in advance for this future optimization.